Data Memo: Detecting Fraudulent Job Postings

Aldo Nunez

October 18, 2026

Contents

Overview of the Dataset 1
Research Question 3
Proposed Timeline 4
Questions and Concerns 5

Overview of the Dataset

For my final project I will use the **Real / Fake Job Posting Prediction** dataset. It contains real-world job advertisements, each labeled as legitimate or fraudulent. The data was originally released by the Laboratory of Information & Communication Systems Security at the University of the Aegean as the *Employment Scam Aegean Dataset* (EMSCAD). The postings were collected between 2012 and 2014.

- **Source:** Kaggle, uploaded by Shivam Bansal
- **Link:** https://www.kaggle.com/datasets/shivamb/real-or-fake-fake-jobposting-prediction
- **Original source:** EMSCAD, University of the Aegean, http://emscad.samos.aegean.gr/

I downloaded the dataset as a single CSV file (`fake_job_postings.csv`, about 50 MB).

```
jobs <- read_csv("fake_job_postings.csv", na = c("", "NA"))
dim(jobs)
```

```
## [1] 17880    18
```

The dataset has **17,880 observations** and **18 columns**. One column is the outcome (`fraudulent`) and one is an ID (`job_id`), which leaves **16 raw predictors**. I plan to engineer more predictors from these, as described below.

Variable types

Type	Variables
Binary (0/1)	`telecommuting`, `has_company_logo`, `has_questions`
Categorical (low/moderate cardinality)	`employment_type`, `required_experience`, `required_education`, `function`
Categorical (high cardinality)	`industry`, `department`, `location` (format: "Country, State, City")
Semi-structured text	`salary_range` (e.g., "50000-65000"), `title`
Free text	`company_profile`, `description`, `requirements`, `benefits`
Outcome	`fraudulent` (0 = real, 1 = fake)

The text fields cannot go directly into most models, so I will turn them into numeric and categorical features. Planned features include:

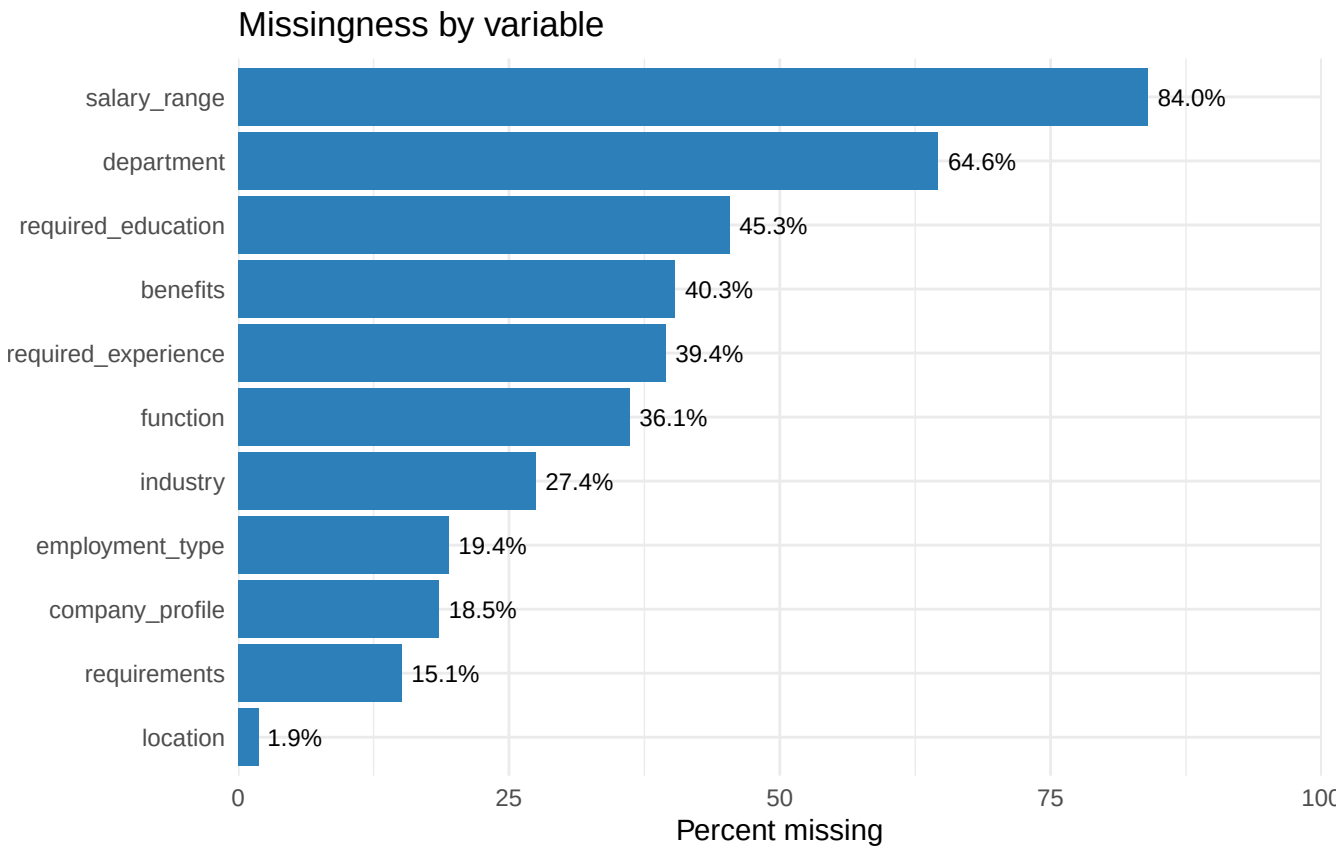
- Whether each text field is present
- Character and word counts
- Simple keyword or TF-IDF features from the description
- Country and state split out from `location`
- Parsed minimum and maximum salary

Missing data

Missing data is a major feature of this dataset. Blank strings were read in as `NA`.

```
missing_tbl <- jobs %>%
  summarise(across(everything(), ~ mean(is.na(.)) * 100)) %>%
  pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") %>%
  filter(pct_missing > 0) %>%
  arrange(desc(pct_missing))

ggplot(missing_tbl, aes(x = reorder(variable, pct_missing), y = pct_missing)) +
  geom_col(fill = "#2c7fb8") +
  geom_text(aes(label = sprintf("%.1f%%", pct_missing)), hjust = -0.15, size = 3.3) +
  coord_flip() +
  scale_y_continuous(limits = c(0, 100), expand = c(0, 0)) +
  labs(x = NULL, y = "Percent missing",
       title = "Missingness by variable") +
  theme_minimal()
```



11 of the 17 non-ID columns have missing values. Missingness ranges from about 2% (`location`) to 84% (`salary_range`). The binary indicators, `title`, `description`, and the outcome are complete.

I don't think this missingness is random. A scammer may well leave out a company profile or details about the role. A quick check supports this:

```
jobs %>%
  mutate(company_profile = if_else(is.na(company_profile), "Missing", "Present")) %>%
  group_by(company_profile) %>%
  summarise(n = n(), fraud_rate = mean(fraudulent)) %>%
  knitr::kable(digits = 3, col.names = c("Company profile", "n", "Fraud rate"))
```

Company profile	n	Fraud rate
Missing	3308	0.177
Present	14572	0.019

Because missingness itself appears to carry signal, I plan to handle it as follows:

- **Categorical variables** (`department`, `industry`, `function`, `employment_type`, `required_experience`, `required_education`): recode `NA` as an explicit `"Not listed"` level instead of imputing or dropping rows.
- **Free-text fields** (`company_profile`, `benefits`, `requirements`): create a "field is present" indicator and a length feature, with length set to 0 when the field is missing.
- **`salary_range`**: with 84% missing, I will mainly use a "salary listed" indicator. I may also add parsed salary bounds, imputed or set to an "unknown" value.
- **`location`**: split into country/state/city. The few missing values become `"Unknown"`.
- Dropping incomplete rows is not an option. It would discard most of the data and likely remove a disproportionate share of the fraudulent postings.

Research Question

Can I predict whether an online job posting is fraudulent from its content and metadata? A second question is which characteristics of a posting are most associated with fraud.

- **Outcome variable:** `fraudulent`, a binary indicator. A value of 1 means the posting was identified as a scam and 0 means it was legitimate.
- **Approach:** This is a **binary classification** problem.

The classes are highly imbalanced:

```
jobs %>%
  count(fraudulent) %>%
  mutate(pct = n / sum(n) * 100) %>%
  knitr::kable(digits = 1, col.names = c("Fraudulent", "n", "Percent"))
```

Fraudulent	n	Percent
0	17014	95.2
1	866	4.8

Only about 4.8% of postings are fraudulent. Accuracy is therefore a poor metric, since a model that always predicts "real" scores about 95%. I will evaluate models mainly with **ROC AUC**, and also report sensitivity and precision-recall. I will use **stratified** train/test splits and stratified cross-validation, and I will try resampling (up- or down-sampling, or SMOTE) inside the recipe.

Predictors I expect to be useful

- `has_company_logo`: legitimate companies are more likely to have a logo on file.
- Missingness indicators, especially for `company_profile`, `requirements`, and `salary_range`.
- `telecommuting` and `has_questions`, since remote jobs without screening questions seem like a common scam pattern.
- `industry` and `function`. I suspect some areas, such as oil and energy or administrative and clerical roles, have higher fraud rates.
- Location (country/state).
- `employment_type`, `required_experience`, and `required_education`.
- Text-derived features: description length and possibly keyword or TF-IDF features for words like "earn," "cash," or "easy."

Goal of the model

The goal is **mainly predictive**. I want a classifier that would flag likely scams on unseen postings, as a job board might use to screen listings. There is also a **descriptive** part: I want to identify which features of a posting are most associated with fraud, using variable importance from tree-based models and coefficients from logistic regression. I am not aiming for formal inference, since the data is observational and from a single job board over a fixed period.

Candidate models

I plan to compare logistic regression (possibly with lasso/elastic net), k-nearest neighbors, a random forest, and a boosted tree model. I will tune them with cross-validation in `tidymodels`.

Proposed Timeline

Week	Dates	Goals
4	Oct 19 – Oct 25	Data loaded (done). Clean `location` and `salary_range`, recode missing values, create text-length and presence features.
5	Oct 26 – Nov 1	Exploratory data analysis: outcome distribution, fraud rates by category, correlation plot of numeric features, visualizations of missingness vs. fraud.
6	Nov 2 – Nov 8	Finalize the feature set. Stratified train/test split, cross-validation folds, and recipe (dummy encoding, lumping rare levels, resampling).
7	Nov 9 – Nov 15	Fit and tune the baseline models (logistic regression, elastic net, KNN).
8	Nov 16 – Nov 22	Fit and tune the tree-based models (random forest, boosted trees). Compare cross-validated ROC AUC.
9	Nov 23 – Nov 29	Thanksgiving week buffer. Select the best model, fit it to the full training set, and evaluate on the test set.

Week	Dates	Goals
10	Nov 30 – Dec 6	Variable importance and interpretation. Write and polish the final report.
Finals	Dec 7 – Dec 11	Final proofreading, knit, and submit.

Questions and Concerns

Anticipated difficulties:

1. **Class imbalance.** With under 5% positives, models may learn to predict "real" almost always. I will need to choose metrics and resampling strategies carefully, and resampling must stay inside cross-validation to avoid leakage.
2. **Text data.** Several of the most informative fields are free text. Turning them into useful predictors without going too deep into NLP is a judgment call. I plan to keep it simple with lengths, indicators, and maybe a small number of TF-IDF terms.
3. **High-cardinality categoricals.** `location` (3,105 unique values), `department` (1,337), and `industry` (131) have many levels. I will lump infrequent levels into an "other" category so dummy encoding stays manageable.
4. **Messy fields.** Some `salary_range` values appear to have been mangled into dates (e.g., "9-Dec"; 26 such values), and `location` is inconsistently formatted. Both will need cleaning.
5. **Generalizability.** The data covers 2012–2014 from one source, so scam patterns may have changed since then.

Questions for the instructional team:

- For the imbalanced outcome, is ROC AUC an appropriate primary metric, or would you prefer we report something like PR AUC or F1?